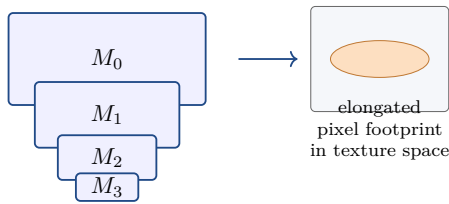


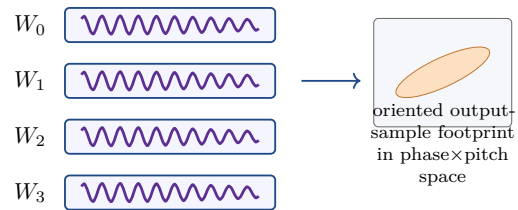
Anisotropic Audio Filters

What DSP programmers can remember from GPU programmers

GPU texture pyramid



Audio wavetable ladder



You already know this sound. Play a saw wave with fast vibrato in any wavetable synth. Listen to the timbre: it *pumps*—brighter and duller twice per vibrato cycle, even when the pitch deviation is only a semitone. That shimmer is **brightness pumping**, and it happens because your synth is doing trilinear filtering where it should be doing anisotropic filtering. This paper explains why, and how to fix it for about two extra multiplies per sample.

Abstract

Every demoscener who has written a wavetable oscillator or tuned a tracker engine has heard brightness pumping: a timbral flutter during vibrato where harmonics surge and recede as the playback cursor sweeps between bandlimited table slices. The standard fix—blending between two adjacent tables, the audio equivalent of trilinear mip filtering—treats pitch as a scalar level-of-detail coordinate. But during fast vibrato, portamento, or any rapid pitch motion, the output sample’s footprint in phase×pitch space is not a circle. It is an *elongated, slanted ellipse*, exactly like a pixel footprint on a texture viewed at a steep angle.

GPU programmers solved this decades ago with anisotropic texture filtering. This paper applies the same mental model to audio: treat the wavetable bank as a 2D source field, estimate the oriented footprint from pitch derivatives, and gather accordingly. Three implementation perspectives are given: an analytical ellipse formulation, a cheap sheared-separable approximation that bolts onto existing sinc interpolators, and procedural pseudocode ready to drop into a tracker engine.

1. The sound you already know

Load a module with vibrato. Any classic S3M or IT will do—Purple Motion, Skaven, pick your favorite. Solo a lead channel with effect Hxx or 4xx (vibrato) at moderate depth and speed. Now listen carefully: the *pitch* wobbles as intended, but the *timbre* wobbles too. The sound gets brighter when the pitch slides up and duller when it slides down, twice per vibrato cycle. This is **brightness pumping**—sometimes called *timbral flutter* or *table-crossfade shimmer*.

The artifact is especially audible on:

- bright waveforms (saws, pulse waves, rich PCM samples),
- fast vibrato speeds (H8x–HFx range),
- deep vibrato depths (Hx8–HxF range),
- portamento slides (Gxx, Exx, Fxx) across several semitones.

The cause is simple. Most wavetable oscillators—from Serum and Vital to libopenmpt’s sinc interpolator—pick one or two prefiltered table slices based on the *current* pitch and blend between them. That is a scalar level-of-detail choice: one blur radius, just like trilinear mip filtering on a GPU. During fast pitch motion, the output sample actually corresponds to a *swath* of pitch values, not a single point. One scalar blur level is not enough.

GPU programmers recognized this exact problem in texture sampling and solved it with *anisotropic filtering*: instead of choosing one mip level, estimate the *oriented footprint* of the pixel in texture space and gather along its major axis. This paper applies that idea to audio.¹

¹The phrase *audio anisotropic filter* is proposed here as a synthesis term, not claimed as established terminology. The underlying DSP ingredients—wavetable mipping, fractional-delay filters, sample-rate conversion—are all well known.[3, 4] The contribution is the *mental model*: treating oscillator antialiasing during pitch motion as the direct analogue of GPU anisotropic texture filtering.[1, 2]

2. The analogy at a glance

If you have ever set `GL_TEXTURE_MAX_ANISOTROPY_EXT` on a texture, you already understand the core idea. Here is the complete mapping:

GPU graphics	Audio synthesis	What it means
Texture coordinates (u, v)	Phase ϕ + pitch-band index μ	Where you read from in sampled source space
Mip pyramid	Prefiltered wavetable bank (bandlimit ladder)	Multiple preblurred copies of the same source
Bilinear inside one mip	Interpolation inside one table slice (linear, cubic, sinc)	In-level reconstruction
Trilinear across two mips	Blend between two neighboring pitch-band tables	Scalar LOD smoothing
Pixel footprint in texture space	Output-sample footprint in phase $\times$ pitch space	The region that <i>really</i> ought to be averaged
Screen-space texture derivatives	$d\phi/dn$, $d\mu/dn$ (optionally $d^2\mu/dn^2$)	Local motion that sizes and orients the footprint
Anisotropic filtering	Oriented gather across nearby phase taps <i>and</i> pitch slices	Handles long, narrow, slanted footprints correctly

The compact teaching statement:

Audio trilinear filtering says: “pick two adjacent bandlimit tables and blend.”

Audio anisotropic filtering says: “estimate the oriented footprint of this output sample in phase $\times$ pitch space, and gather accordingly.”

3. What you are already doing

Before diving into the math, it helps to see where existing techniques fall on the isotropic $\leftrightarrow$ anisotropic spectrum.

PolyBLEP / BLIT. These are point-evaluated antialiasing methods for hard-sync and waveform discontinuities. They handle aliasing at the current pitch but have no concept of pitch *motion*—they operate sample-by-sample with no directional awareness. Fully isotropic.

Serum / Vital wavetable mipping. Modern soft synths precompute a ladder of bandlimited tables and crossfade between adjacent slices based on pitch. This is textbook “audio trilinear”—a scalar LOD blend, excellent at steady pitch, vulnerable to brightness pumping during modulation.

Tracker SINC interpolation (libopenmpt, XMPlay, OpenMPT). High-quality phase interpolation (typically 8–16 taps, Kaiser-windowed), but pitch selection is still a single-point lookup. The interpolation is superb along the phase axis and nonexistent along the pitch axis. It is like running $16\times$ anisotropic filtering in u but only nearest-neighbor in v .

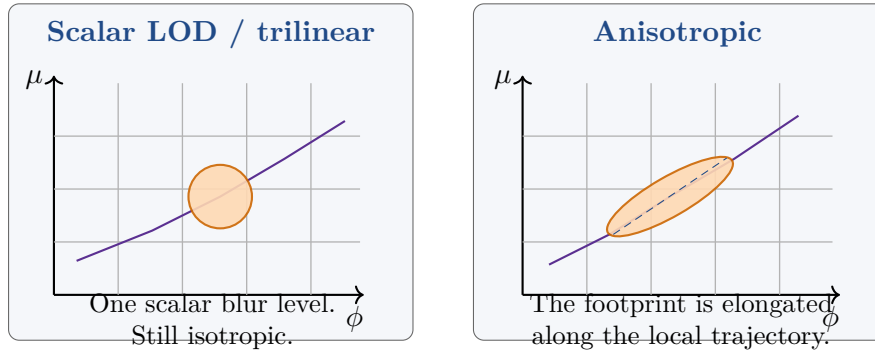


Figure 1: The core mental-model shift. Left: scalar LOD blending picks one blur radius—isotropic. Right: the footprint is elongated along the pitch-motion trajectory—anisotropic. The playback cursor traces a slanted path through phase×pitch space during vibrato.

The gap. None of these methods *orient* the reconstruction kernel according to local pitch motion. That is the gap this paper fills.

4. The GPU mental model

4.1 Mipmaps, scalar LOD, and trilinear filtering

A GPU texture is accompanied by a pyramid of prefiltered images called *mip levels*. A pixel shader wants the average value of the texture under the pixel’s footprint mapped into texture space. Mipmapping approximates that by choosing a blur level whose scale roughly matches the footprint, and trilinear filtering smooths the transition between adjacent levels.[1, 2]

Compactly:

$$C \approx \text{lerp}(\text{bilinear}(M_\ell, u, v), \text{bilinear}(M_{\ell+1}, u, v), \text{frac}(\lambda)), \quad (1)$$

where M_ℓ and $M_{\ell+1}$ are adjacent mip levels and λ is a scalar level-of-detail.

This works well when a *single scalar blur level* is a good description of the footprint.

4.2 Why anisotropic filtering exists

The scalar LOD picture breaks down when the footprint in texture space is long, narrow, and slanted—for example, when a textured floor recedes into the distance. Trilinear filtering blurs too much along one axis and not enough along the other. Anisotropic filtering acknowledges that the footprint has *direction*, *aspect ratio*, and *orientation*.[2]

- **Trilinear:** estimate one blur radius, choose two mip levels, blend.
- **Anisotropic:** estimate an *oriented footprint*, gather in a way that respects its major axis.

That exact distinction carries into sampled synthesis.

5. The direct audio analogy

5.1 The wavetable bank is a texture pyramid

Let

$$W(\phi, \mu) \tag{2}$$

be a 2D source field where ϕ is the **phase coordinate** within one cycle and μ is a **pitch-band index** across prefiltered wavetable slices. This turns a bank of bandlimited wavetables into the audio counterpart of a texture with mip levels. Advancing along ϕ reads successive samples; moving along μ selects progressively more bandlimited tables.[3]

5.2 Audio trilinear filtering

The standard two-table blend is the audio trilinear:

1. reconstruct inside table q at phase ϕ ,
2. reconstruct inside table $q + 1$ at phase ϕ ,
3. blend using the fractional part of the pitch-band coordinate μ .

If $q = \lfloor \mu \rfloor$ and $\varepsilon = \mu - q$, then

$$y[n] \approx (1 - \varepsilon_n)S_q(\phi_n) + \varepsilon_n S_{q+1}(\phi_n), \tag{3}$$

where each S_q may itself be reconstructed by linear, cubic, or windowed-sinc interpolation.

This is already a useful method. But it is still a *scalar level blend*. It assumes that one blur coordinate is enough.

5.3 Why vibrato exposes the limitation

During steady pitch, the playback path is nearly axis-aligned in phase×pitch space. During fast vibrato or glissando, it becomes slanted. The output sample corresponds not to a point or a scalar LOD, but to an *oriented local neighborhood* around the playback trajectory.

That is the exact point where the graphics analogy becomes most informative—and where brightness pumping originates.

6. Definition

An **audio anisotropic filter** for sampled synthesis is a time-varying reconstruction operator that:

1. treats the source as a field over at least two coordinates (phase ϕ and pitch-band μ),
2. estimates a local playback footprint from derivatives such as $d\phi/dn$ and $d\mu/dn$,
3. allows the reconstruction kernel to be *oriented* and *stretched*, not only isotropic,

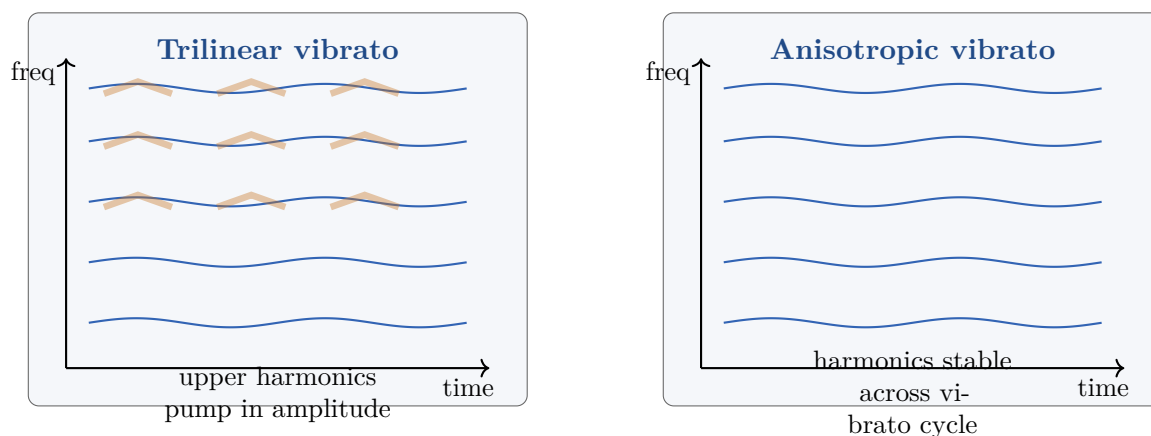


Figure 2: Schematic spectrograms during vibrato. Left: trilinear table blending causes upper harmonics to pump in amplitude as the crossfade sweeps between table slices—this is brightness pumping. Right: anisotropic gathering along the pitch trajectory keeps harmonic amplitudes stable. Both have identical pitch modulation.

4. gathers from an oriented neighborhood in source space instead of a single table or a simple two-table blend.

In one sentence:

Audio anisotropic filtering is to a wavetable mip ladder what GPU anisotropic filtering is to a texture mip ladder.

7. Why it is useful

7.1 Pitch motion is the primary use case

The most immediate use cases are vibrato, glissando, portamento, pitch envelopes, and any time-varying resampling. The literature makes clear that wavetable playback quality is tied to sample-rate-conversion quality and that anti-aliasing should vary with conversion ratio.[3, 4]

The anisotropic viewpoint adds a stronger statement:

When the local conversion ratio is changing, the reconstruction footprint should tilt and stretch according to that motion.

7.2 What you gain

- **Less alias leakage during fast pitch motion.** The kernel is conservative over the whole footprint, not just at the center pitch.
- **Reduced brightness pumping.** Neighboring tables contribute according to geometry, not only a scalar crossfade.

- **More stable timbre under vibrato.** A two-table blend alternates between too sharp and too dull; an oriented gather is steadier.
- **One reconstruction story.** The problem stops looking like several ad hoc heuristics and starts looking like one operator over a sampled field.

8. Analytical formulation

8.1 Source field and playback trajectory

Think of the wavetable bank as a 2D surface. The playback cursor traces a path across it:

$$W : \mathbb{R}^2 \rightarrow \mathbb{R}, \quad W(\phi, \mu), \quad (4)$$

$$\mathbf{c}_n = \begin{bmatrix} \phi_n \\ \mu_n \end{bmatrix}. \quad (5)$$

For a vibrato oscillator, ϕ_n advances each sample and μ_n oscillates with the modulation.

8.2 Continuous oriented-footprint form

The anisotropic filter is a weighted average over an oriented neighborhood—the audio EWA (elliptical weighted average), directly analogous to GPU EWA texture filtering:

$$y[n] = \frac{\iint_{\mathbb{R}^2} W(\mathbf{u}) K_n(\mathbf{u} - \mathbf{c}_n) d\mathbf{u}}{\iint_{\mathbb{R}^2} K_n(\mathbf{u} - \mathbf{c}_n) d\mathbf{u}}, \quad \mathbf{u} = \begin{bmatrix} \phi \\ \mu \end{bmatrix}. \quad (6)$$

The kernel is defined by an ellipse metric that encodes the footprint’s shape and orientation:

$$K_n(\Delta) = F\left(\Delta^T \Sigma_n^{-1} \Delta\right), \quad (7)$$

where Σ_n is a positive definite 2×2 matrix controlling scale and orientation.

8.3 Ellipse from local derivatives

The ellipse shape comes directly from how fast the cursor is moving through phase×pitch space. Let

$$\mathbf{v}_n = \begin{bmatrix} \dot{\phi}_n / s_\phi \\ \dot{\mu}_n / s_\mu \end{bmatrix}, \quad (8)$$

where s_ϕ and s_μ rescale phase and pitch-band units into comparable geometric units. Let $\mathbf{t}_n = \mathbf{v}_n / \|\mathbf{v}_n\|$ be the local direction of travel, and let R_n be the rotation that maps the horizontal axis to $\mathbf{t}_n$. Then:

$$\Sigma_n = R_n \begin{bmatrix} \sigma_{\parallel,n}^2 & 0 \\ 0 & \sigma_{\perp,n}^2 \end{bmatrix} R_n^T. \quad (9)$$

The major axis $\sigma_{\parallel,n}$ grows when pitch motion is fast; the minor axis $\sigma_{\perp,n}$ stays close to the steady-pitch reconstruction width:

$$\sigma_{\parallel,n} = \sigma_0 + k_v \|\mathbf{v}_n\|, \quad \sigma_{\perp,n} = \sigma_{\text{base}}, \quad (10)$$

with optional dependence on acceleration $\ddot{\mu}_n$ for very rapid bends.

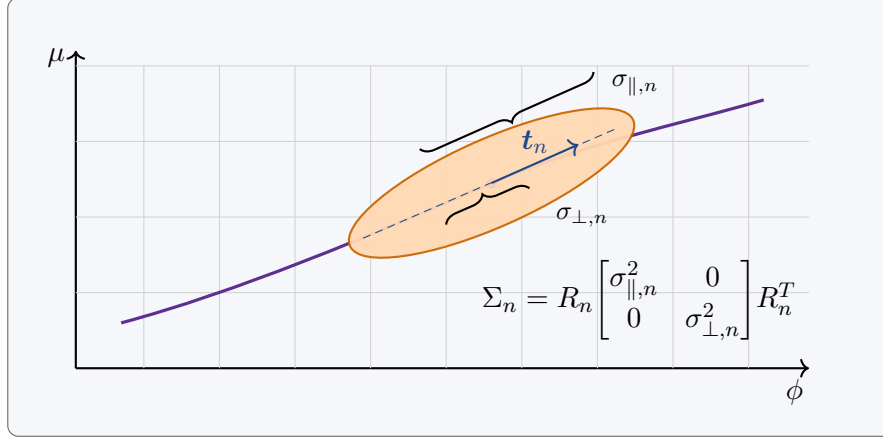


Figure 3: EWA view in phase×pitch space. The output sample is reconstructed from an oriented elliptical neighborhood aligned with the local trajectory $\mathbf{t}_n$. During fast vibrato, $\sigma_{\parallel}$ stretches; at vibrato turning points it relaxes toward isotropic.

8.4 An anisotropic sinc kernel

For a more explicitly bandlimited flavor than a Gaussian, define the kernel radius in ellipse coordinates and apply a windowed sinc:

$$r_n(\Delta) = \sqrt{\Delta^T \Sigma_n^{-1} \Delta}, \quad (11)$$

$$K_n(\Delta) = \text{sinc}(\alpha r_n(\Delta)) w(r_n(\Delta)), \quad (12)$$

where $w(\cdot)$ is a compact window (Kaiser, Blackman, etc.). This is the most direct mathematical statement of an *anisotropic sinc kernel*: a radial sinc written in ellipse coordinates instead of circle coordinates.

8.5 Discrete gather form

In practice, evaluate only a local neighborhood. If $m_0 = \lfloor \phi_n \rfloor$ and $q_0 = \lfloor \mu_n \rfloor$:

$$y[n] \approx \frac{\sum_{i=-P}^P \sum_{j=-L}^L W[q_0 + i, m_0 + j] K_n \left(\begin{bmatrix} j - \delta_n \\ i - \varepsilon_n \end{bmatrix} \right)}{\sum_{i=-P}^P \sum_{j=-L}^L K_n \left(\begin{bmatrix} j - \delta_n \\ i - \varepsilon_n \end{bmatrix} \right)}, \quad (13)$$

where $\delta_n = \text{frac}(\phi_n)$ and $\varepsilon_n = \text{frac}(\mu_n)$.

9. A sheared separable approximation

The full ellipse form is conceptually clean but expensive. For a real oscillator, the practical path is a *sheared separable kernel* that bolts directly onto an existing sinc interpolator. Instead of evaluating the full ellipse metric, let pitch motion shear the phase kernel:

$$K_n(\Delta\phi, \Delta\mu) = \text{sinc}(c_\phi(\Delta\phi - \beta_n \Delta\mu)) w_\phi \left(\frac{\Delta\phi - \beta_n \Delta\mu}{L} \right) \cdot \text{sinc}(c_\mu \Delta\mu / R_n) w_\mu \left(\frac{\Delta\mu}{P R_n} \right). \quad (14)$$

The two free parameters come directly from pitch motion:

$$\beta_n = k_\beta \dot{\mu}_n, \quad R_n = 1 + k_R |\dot{\mu}_n|. \quad (15)$$

In plain English:

- β_n **tilts** the phase taps according to the direction of pitch travel,
- R_n **widens** support across pitch slices when pitch is moving quickly,
- the phase kernel remains sinc-like, so this can be implemented as a modified SINC-16 or similar.

When $\dot{\mu}_n = 0$ (steady pitch), the kernel reduces to ordinary separable interpolation. During vibrato, it becomes direction-aware.

9.1 Cost

The sheared approximation is cheap. Here is how it compares to what you are probably already doing:

Method	Tables	Taps	Muls/smp	Adds/smp
SINC-16 (phase only)	1	16	16	16
Trilinear (2×SINC-8)	2	16	17	17
Sheared aniso (2×SINC-8, β -shift)	2	16	~19	~19
Full EWA (3×SINC-8)	3	24	~27	~27

The sheared approximation costs roughly **2 extra multiplies per sample** over trilinear: one for computing the phase shift $\beta_n \Delta\mu$ and one for the pitch-axis window scaling R_n . On modern x86 with FMA, this is negligible.

Quick win: minimum viable anisotropic filter

If you already blend between two wavetable mip levels (trilinear), add one thing: when $d\mu/dn \neq 0$, **shift the phase tap positions** in the upper table by $\beta \cdot \Delta\mu$ where $\beta = k \cdot d\mu/dn$. That single shear turns your trilinear filter into a first-order anisotropic filter. Cost: one multiply and one add per tap. Benefit: brightness pumping during vibrato drops significantly. Start with $k_\beta = 0.5$ and tune by ear. You will hear the difference on any bright waveform with fast vibrato.

10. Procedural implementation

10.1 Offline precomputation

Build a *wavetable mip ladder*, exactly like generating a GPU texture mip pyramid:

1. choose a base waveform,

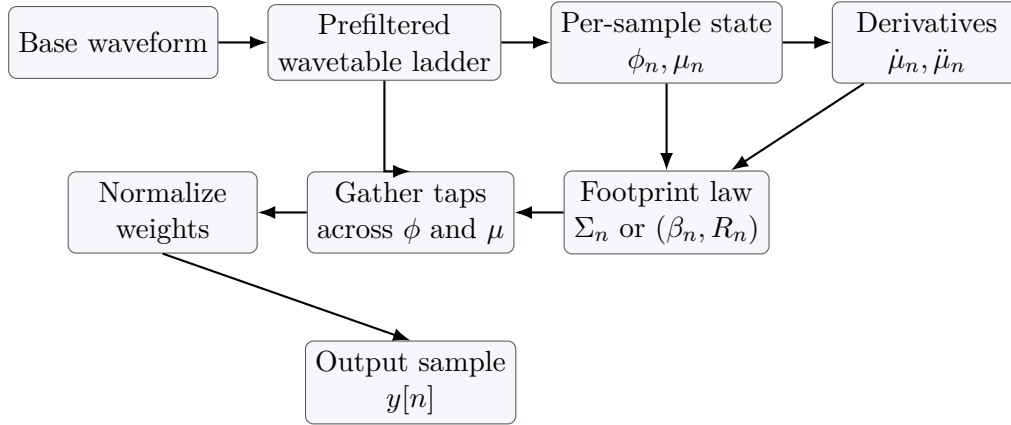


Figure 4: Procedural pipeline. The filter is a gather over a prefiltered wavetable ladder, driven by per-sample estimates of local pitch motion.

2. generate a bank of prefiltered tables $W[q, m]$ with progressively lower harmonic ceilings,
3. map each table index q to a target pitch region or bandlimit,
4. optionally store integrated or preconditioned tables for higher-order interpolation.[3]

10.2 Runtime algorithm

For each output sample:

1. advance the continuous phase ϕ_n ,
2. update the continuous pitch-band coordinate μ_n ,
3. estimate $\dot{\mu}_n$ (and optionally $\ddot{\mu}_n$),
4. construct either an ellipse matrix Σ_n or the cheaper sheared approximation (β_n, R_n) ,
5. gather neighboring taps from nearby phase positions *and* nearby pitch slices,
6. weight them with the anisotropic kernel,
7. normalize and emit the sample.

10.3 Conservative bandlimit choice

A practical detail: the phase low-pass behavior should use the *most conservative pitch within the footprint*, not just the center pitch. If the footprint spans slices $q_0 - P$ through $q_0 + P$, use the cutoff appropriate for the highest-risk slice. This mirrors the sample-rate-conversion fact that anti-alias cutoff depends on the relationship between input and output rates.[3]

10.4 Pseudocode

```

double anisotropic_sample(
    const TableBank& W,
    double phase,           // continuous phase coordinate
    double pitchCoord,      // continuous pitch-band coordinate
    double dPitch,          // first derivative dmu/dn
    Params p)
{
    int m0 = floor(phase);
    int q0 = floor(pitchCoord);
    double delta = phase - m0;
    double eps = pitchCoord - q0;

    double beta = p.kBeta * dPitch;
    double R = 1.0 + p.kR * fabs(dPitch);

    double acc = 0.0;
    double norm = 0.0;

    for (int i = -p.P; i <= p.P; ++i) {
        double dmu = (i - eps) / R;
        double wMu = sinc(p.cMu * dmu)
            * window_pitch(dmu / p.P);

        double phaseShift = beta * (i - eps);

        for (int j = -p.L; j <= p.L; ++j) {
            double dphi = (j - delta) - phaseShift;
            double wPhi = sinc(p.cPhi * dphi)
                * window_phase(dphi / p.L);

            double w = wPhi * wMu;
            acc += w * W[q0 + i][m0 + j];
            norm += w;
        }
    }

    return (norm != 0.0) ? (acc / norm) : 0.0;
}

```

11. A worked vibrato model

Suppose the pitch-band coordinate follows sinusoidal vibrato (the most common modulation in tracker music):

$$\mu(t) = \mu_0 + D \sin(2\pi f_v t). \quad (16)$$

Then its derivatives are:

$$\dot{\mu}(t) = 2\pi f_v D \cos(2\pi f_v t), \quad \ddot{\mu}(t) = -(2\pi f_v)^2 D \sin(2\pi f_v t). \quad (17)$$

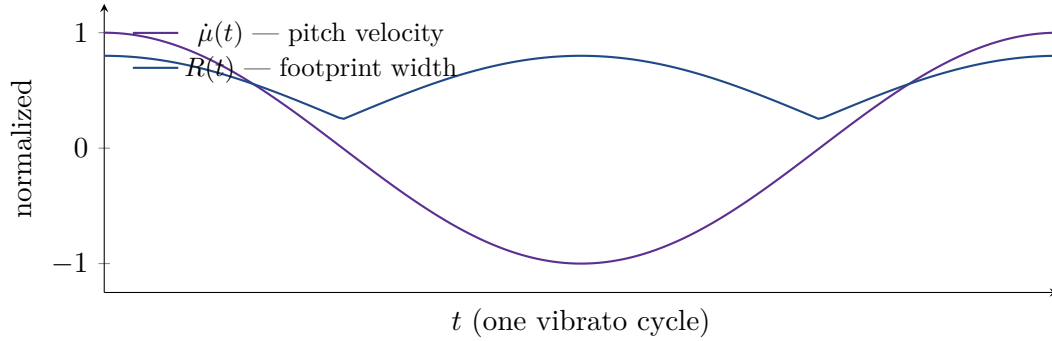


Figure 5: For sinusoidal vibrato, the footprint widens (R grows) when pitch velocity $|\dot{\mu}|$ is large and relaxes toward isotropic at the turning points. The filter is most aggressive exactly when brightness pumping is worst.

Insert these into the sheared approximation:

$$\beta(t) = k_{\beta}\dot{\mu}(t), \quad R(t) = 1 + k_R|\dot{\mu}(t)|. \quad (18)$$

The filter automatically becomes more elongated when the vibrato slope is steep (maximum pitch velocity at zero-crossing) and more isotropic at the turning points where $\dot{\mu}(t) = 0$ (vibrato extremes, where pitch is momentarily stationary). This is exactly where brightness pumping is worst (steep slope) and absent (turning points)—the filter adapts to the modulation shape.

12. Connection to real engines

This is not a purely theoretical exercise. The author’s tracker engine (SUPERNOVA, a vendored libopenmpt fork with a double-precision audio pipeline) currently uses 16-tap Kaiser-windowed sinc interpolation with 16384 phase subpositions and double-precision accumulation throughout the mixer. The sheared anisotropic extension described here is a natural next step: the existing phase kernel provides the infrastructure, and the pitch-axis gather adds only the β -shear and a second table lookup per tap.

The same approach applies to any engine with a multi-table wavetable oscillator or a high-quality sinc interpolator—Serum, Vital, Surge, or a custom softsynth. If you already have the mip ladder, the anisotropic part is a few lines of code on top.

13. Engineering tradeoffs

This viewpoint is useful, but it does not erase practical tradeoffs:

- **Cost.** A true elliptical gather is more expensive than a two-table blend. The sheared approximation exists for exactly this reason.
- **Parameterization.** The units of phase and pitch must be scaled into a common geometry via s_{ϕ} and s_{μ} ; poor scaling yields poor footprints.

- **Table density.** Sparse pitch slices limit how well the method approximates a smooth footprint across μ .
- **Not every oscillator needs it.** At steady pitch, ordinary trilinear table blending is sufficient. The benefit scales with pitch-motion speed.

The method is best thought of as a *geometry-aware extension* of familiar wavetable mip-ladder techniques, not a universal replacement. Use it where it matters: fast vibrato, deep portamento, pitch envelopes, and any context where you can hear the timbre wobble.

14. Summary

An audio anisotropic filter can be described in one sentence:

It is a direction-aware reconstruction filter for sampled synthesis that treats each output sample as an oriented footprint over a phase×pitch wavetable mip ladder, rather than as a point lookup or a scalar blend between two tables.

That is the direct conceptual bridge to GPU anisotropic texture filtering. If you have ever wondered why `GL_TEXTURE_MAX_ANISOTROPY_EXT` makes distant textures look so much better, and you have also wondered why your wavetable vibrato sounds slightly wrong—now you know they are the same problem, and they have the same solution.

References

- [1] Paul S. Heckbert, *Fundamentals of Texture Mapping and Image Warping*, University of California, Berkeley, Technical Report UCB/CSD 89/516, 1989.
[Online PDF](#)
- [2] Khronos Group, *EXT_texture_filter_anisotropic*, OpenGL extension specification.
[Specification](#)
- [3] Andreas Franck and Vesa Välimäki, *Higher-Order Integrated Wavetable Synthesis*, Proceedings of the 15th International Conference on Digital Audio Effects (DAFx-12), 2012.
[Online PDF](#)
- [4] Vesa Välimäki and Timo I. Laakso, *Principles of Fractional Delay Filters*, ICASSP 2000 tutorial paper.
[Online PDF](#)